



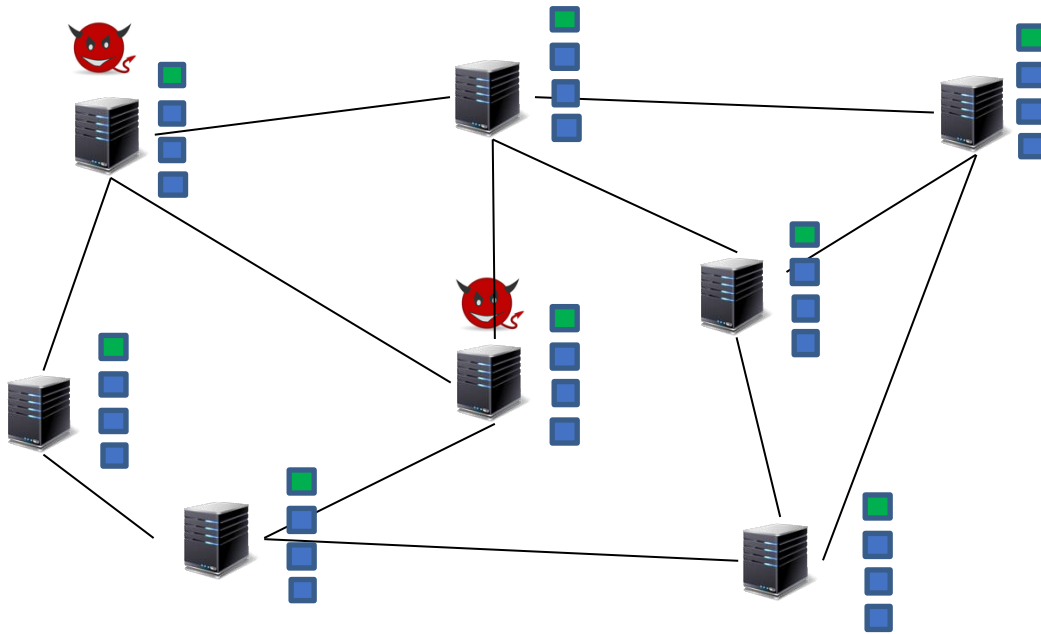
Blockchain Principles and Applications

Amir Mahdi Sadeghzadeh, PhD

Data and Network Security Lab (DNSL)
Trustworthy and Secure AI Lab (TSAIL)

Recap

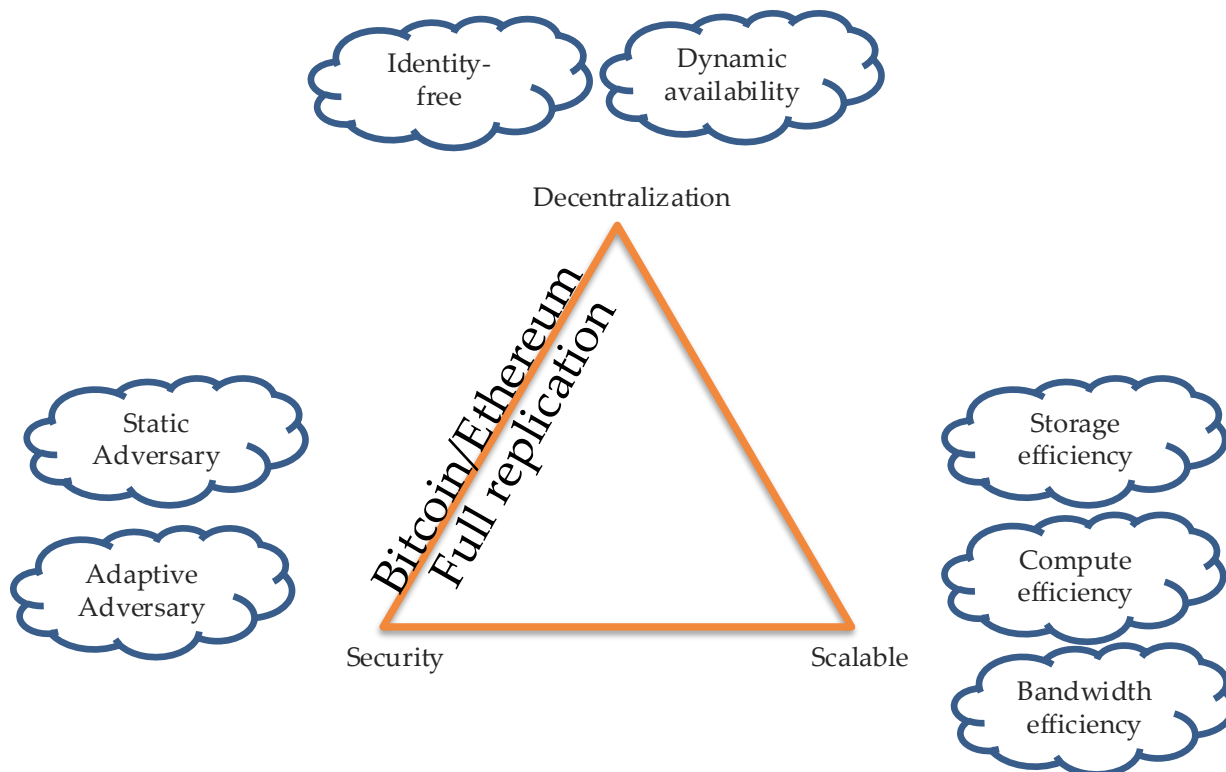
Blockchains & Full replication



Full replication – resource usage

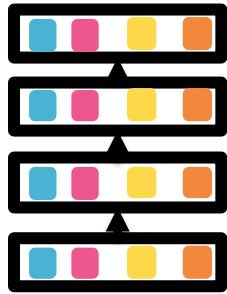
- All nodes process the same transactions
- **Communication:** The transaction has to traverse the complete network at least once
- **Storage:** All nodes have to store the complete state, the account details of everyone!
- **Compute:** All nodes have to validate all transactions and update the ledger every block

Trilemma

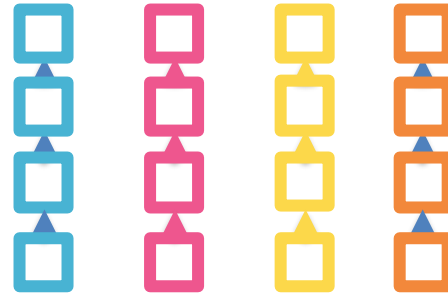


First Approach– Maintain multiple blockchains

- Divide ledger into K shards
- Each shard is a separate blockchain



Complete ledger



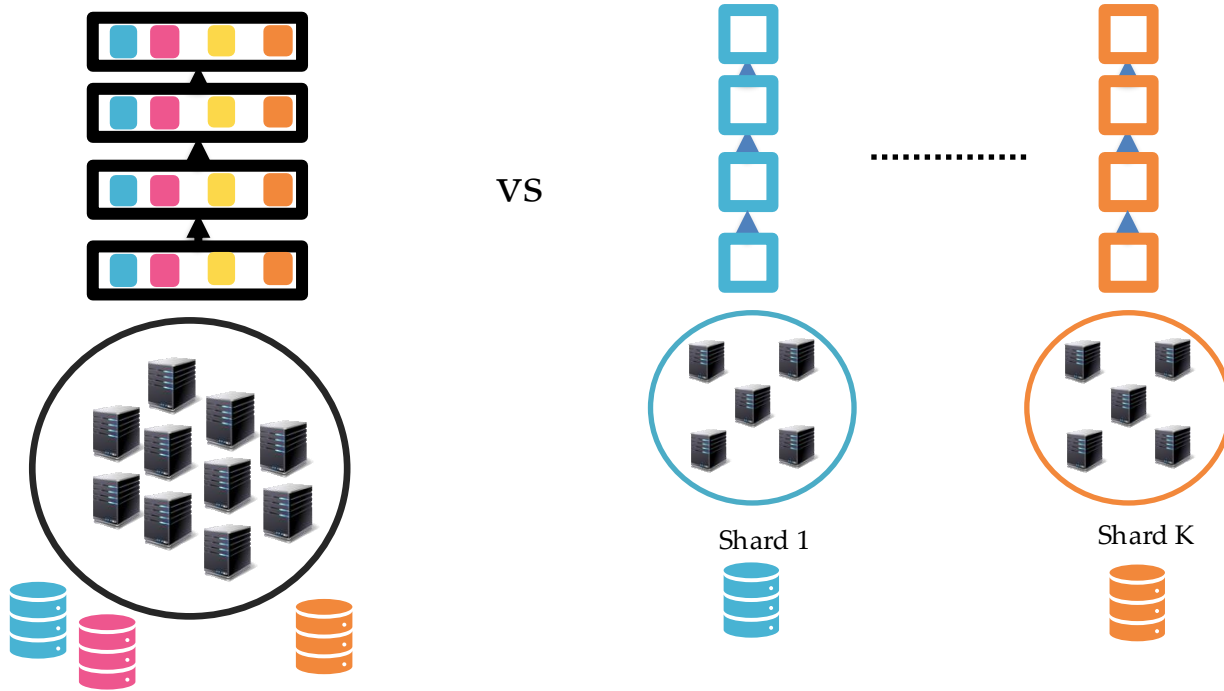
K Shards

Problems

1. Reduced security – An adversary can concentrate on one shard
2. How to transfer money from one shard to another?
3. If such transfer is possible, adversary can transfer non-existent funds from infected shard to non-infected shard.

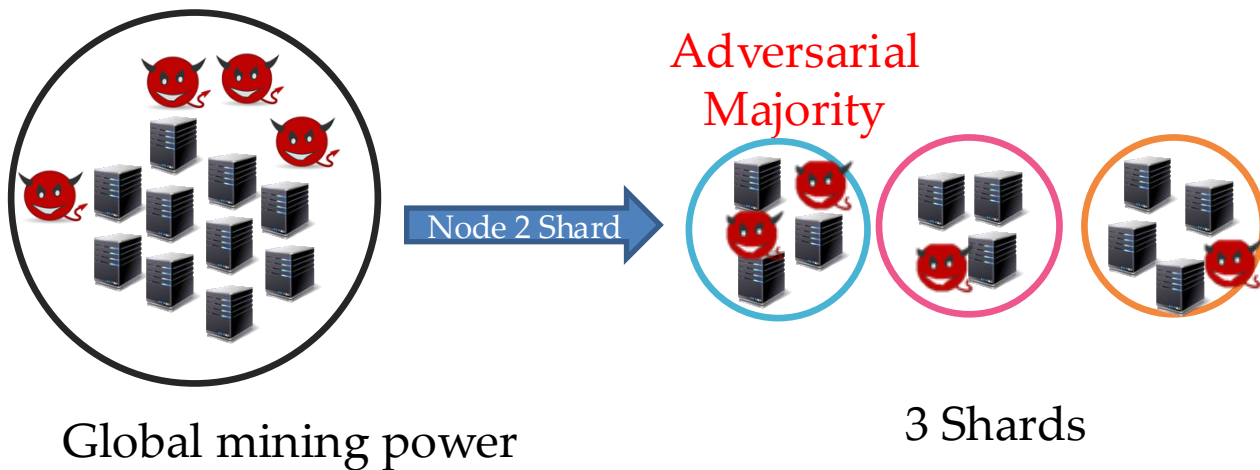
Scaling: $O(K)$

- Nodes only maintain the state of their own shard



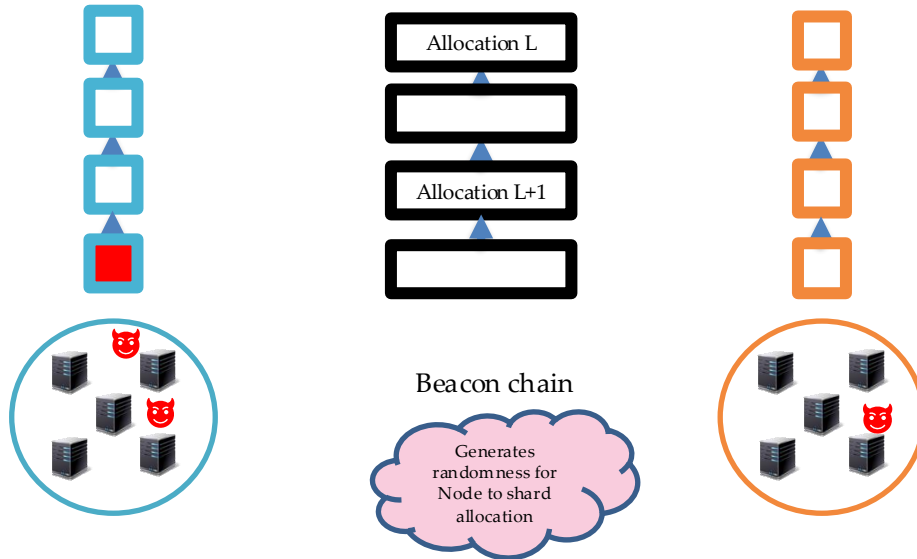
Drawback 1: Proportional representation

- Need large number of nodes per shard to ensure honest majority in a shard



Drawback 2: Security

- Not resilient to $O(1/K)$ adaptive adversary



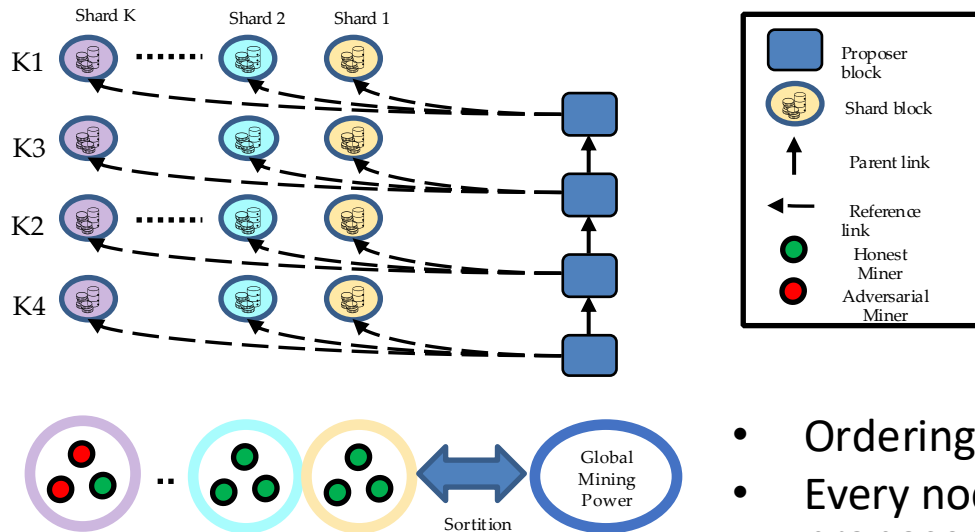
Drawback 3: Node identity

- Existing works vary in the implementation of Node to shard allocation
 - Different ways of randomness generation
 - Different rate of re-allocation
- All Node to shard allocation algorithms require consensus node identity

Identity-free sharding

- If the protocol is identity free, we cannot use Node to Shard allocation algorithm
- Only choice is to allow nodes to **self-allocate**
- Adversaries can congregate on one shard; safety and liveness can be easily broken
- Solution: **Uniconsensus** architecture

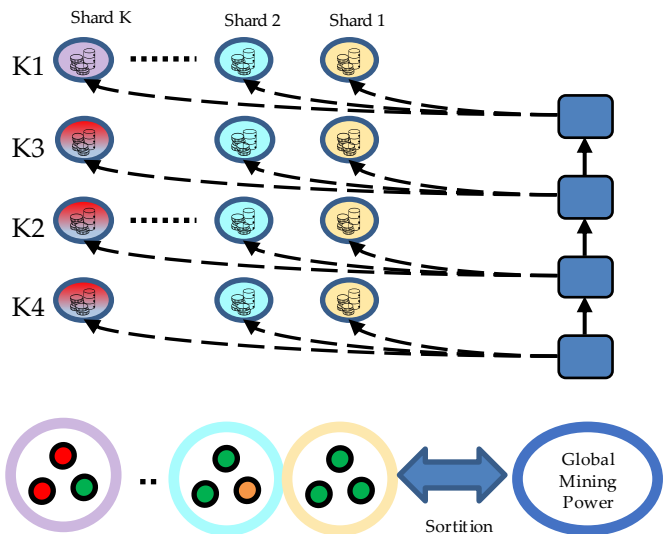
Uniconsensus: Safety



- Ordering: **Proposer chain**
- Every node maintains proposer chain
- Adversarial majority in a shard does not violate safety

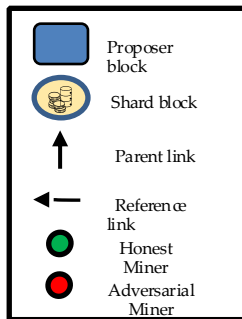
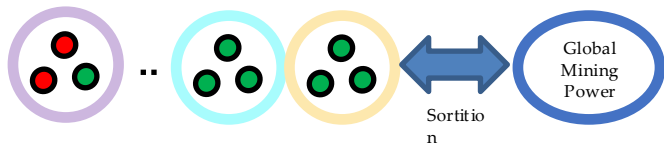
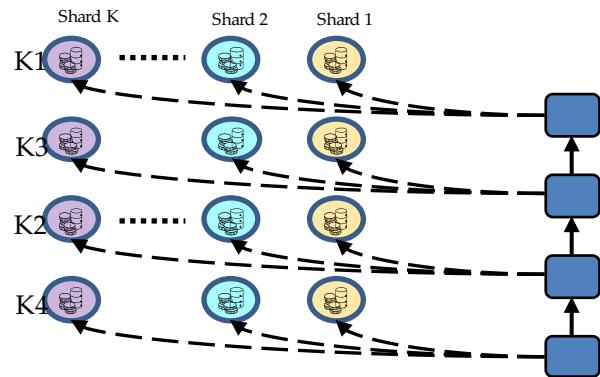
Uniconsensus: Liveness

Chain-quality visualization



- Adversaries congregate: drown out honest miners
- Dynamic self allocation can prevent such attacks

Uniconsensus Architecture



- Safety shared
- Liveness is sacrificed

Sharding (continue)

State commitments: Bootstrapping

- Both Uniconsensus and Multiconsensus depend on nodes relocating to new shards
 - Efficient methods of bootstrapping should be used
1. Download latest state of a shard (instead of the whole ledger)
 2. Verify state from the latest state-commitment
 3. State-commitment is a root of the Merkle tree of execution state

Accumulator for State-commitments

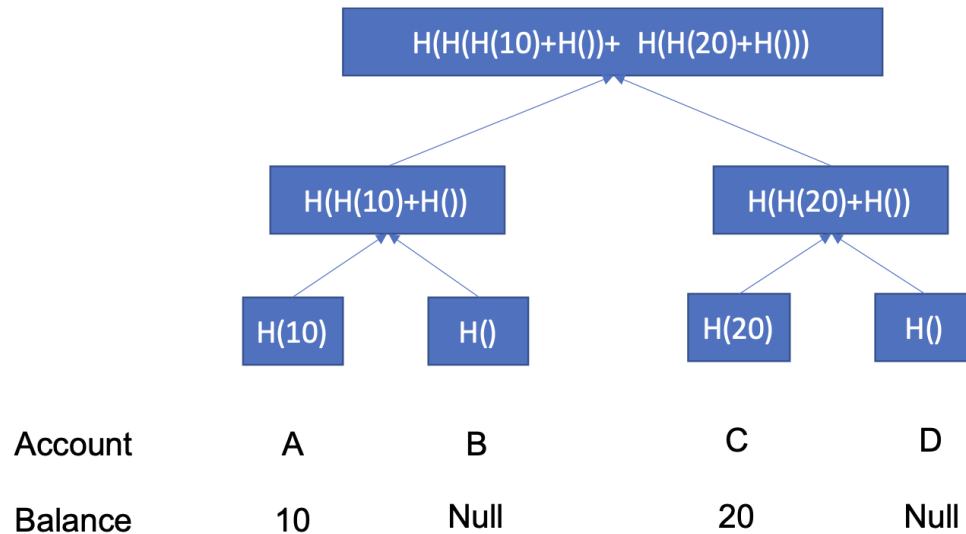


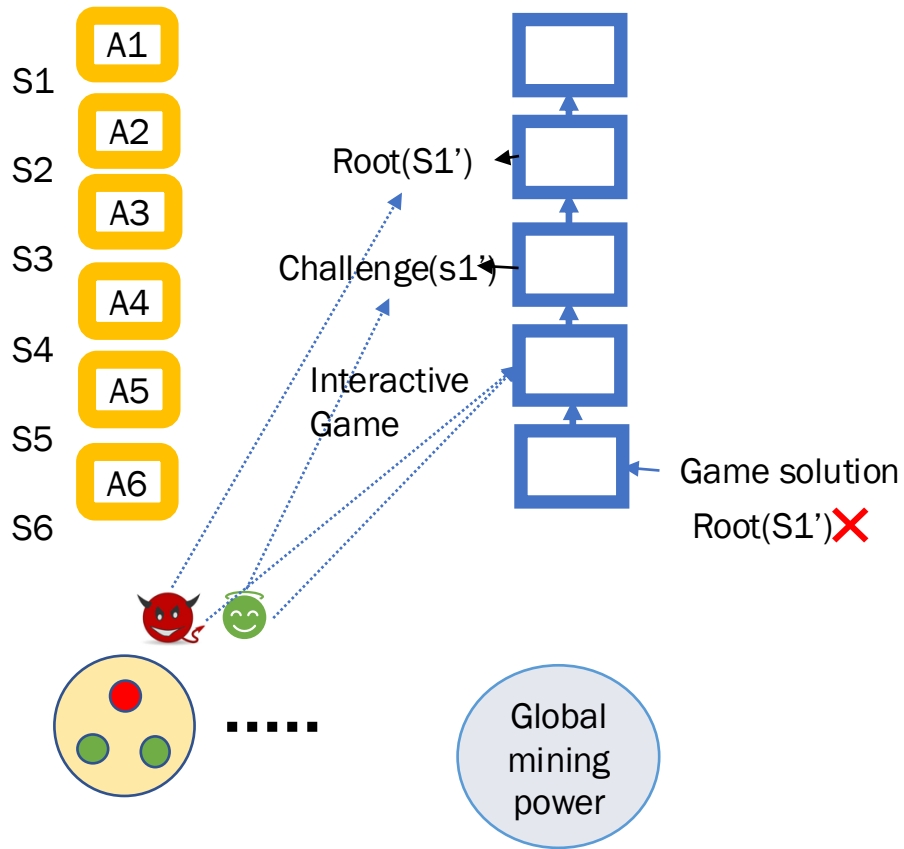
Figure 4: State-commitment using a sparse Merkle tree.

State-commitments: Multiconsensus

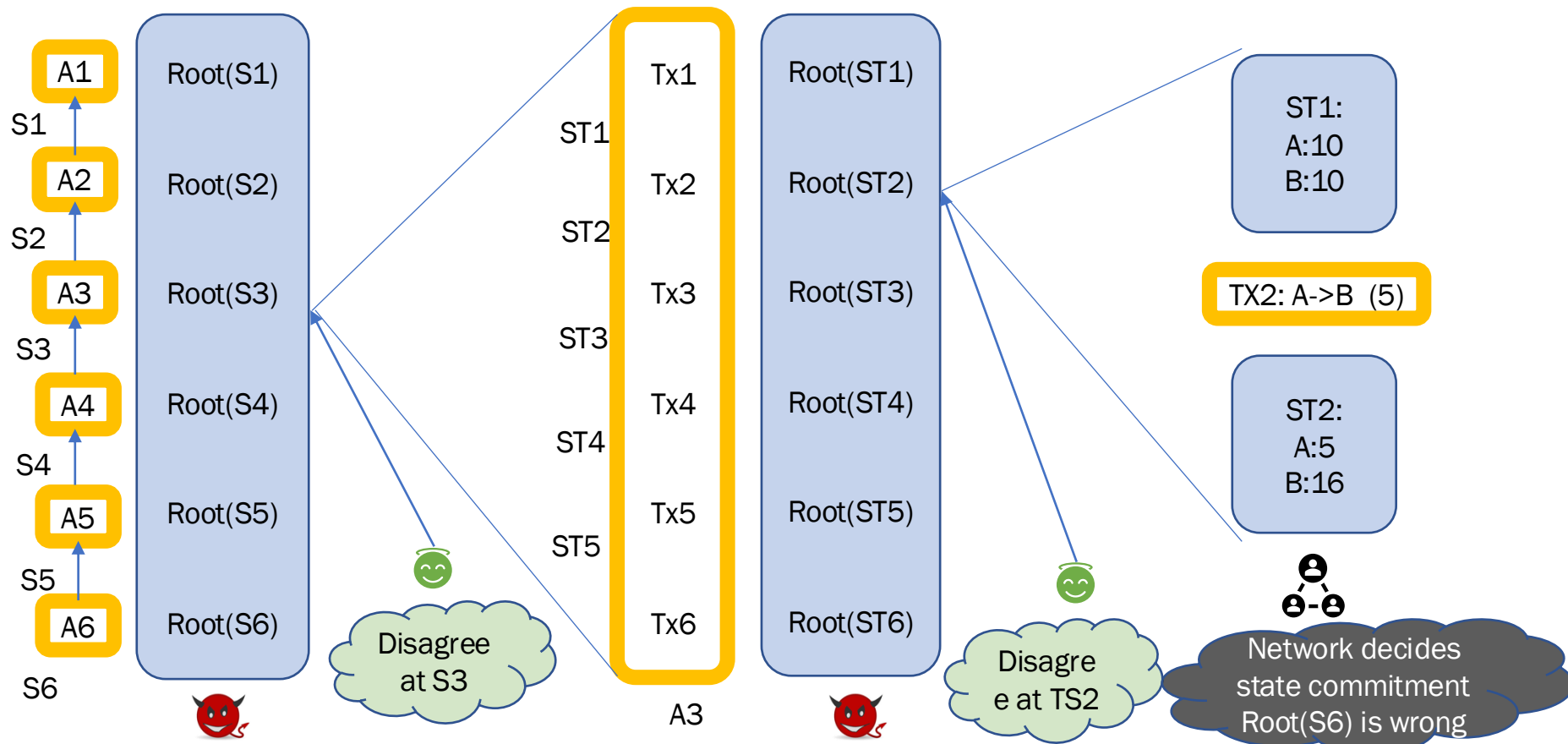
- Assumption: Each shard has honest majority
- Two possible approaches:
 1. A state-commitment is correct if signed by majority in a shard
 2. A state-commitment is correct if a transaction including one is finalized in the shard's ledger

State-commitments: Uniconsensus

- **Problem:** No honest majority assumption and no coupled validation
- **Solution:** Interactive fraud proof mechanism for detecting incorrect state-commitments
- Logarithmic fraud search between two parties: Proposer and Challenger



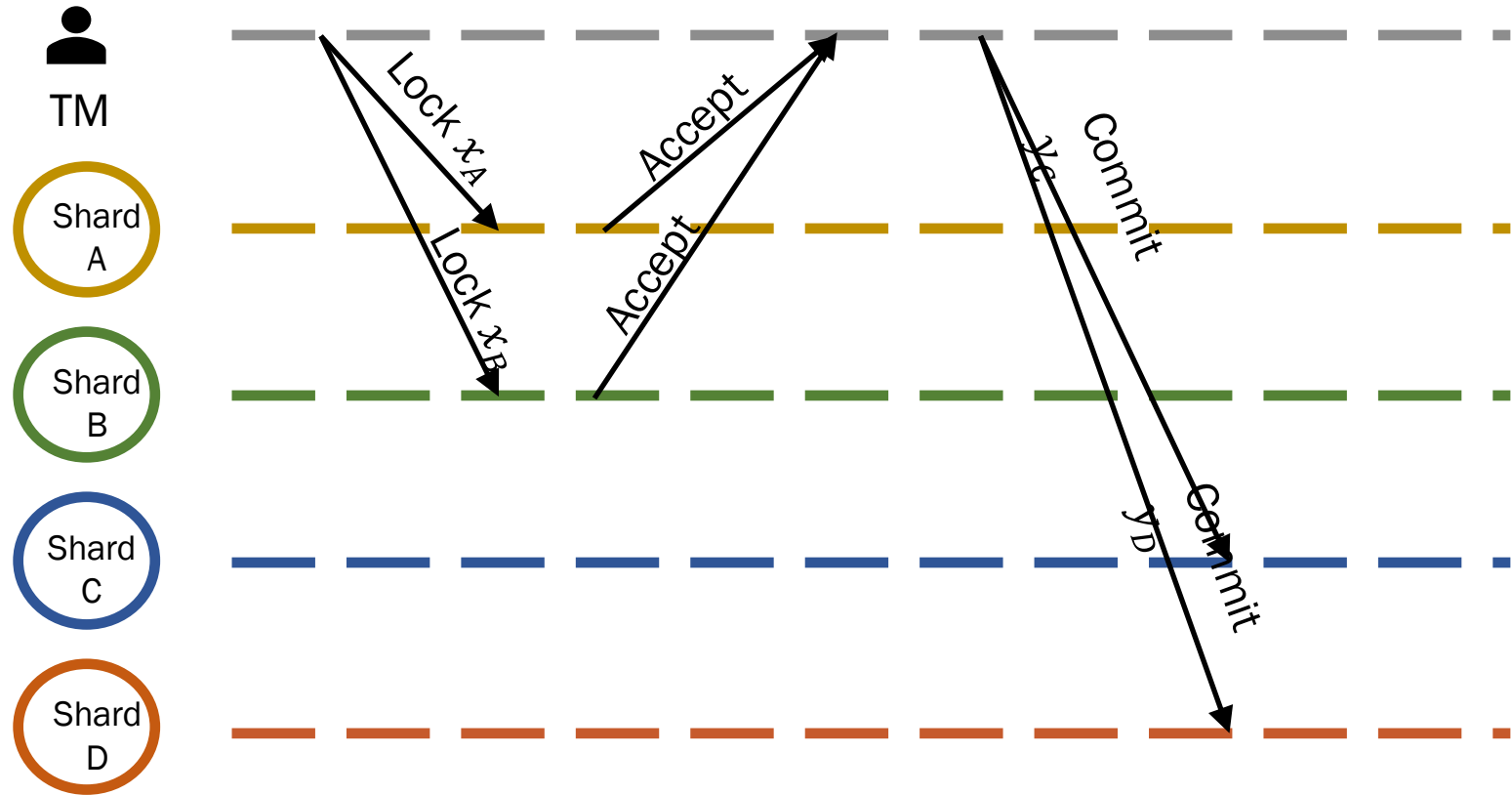
Interactive fraud-proof: Uniconsensus



Cross-shard transactions

- Rely on state-commitments for a foreign-shard's state
- Or rely on a majority signature from foreign-shard's committee
- Atomicity: What if a transactions has multiple input-shards and multiple output-shards
- Atomicity: Transactions have to be cancelled on either all of the shards or none of the shards
- 2-Phase commits (2PC) are used to preserve atomicity

2-Phase Commits

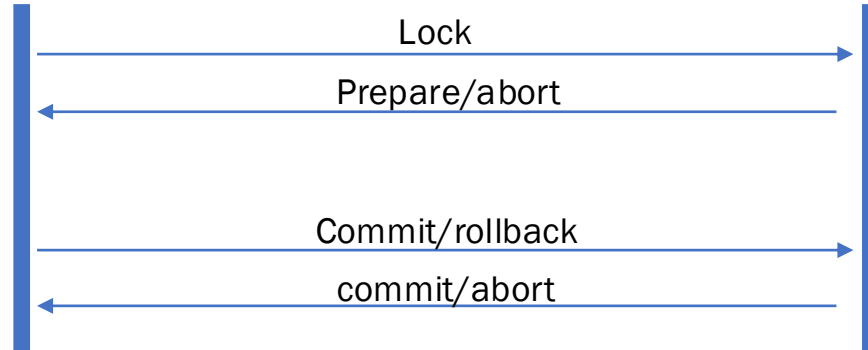


Two phase commit/alternate

Transaction Manager



Shards



Conclusion

- Sharding fell out of fashion
 - Changes at the consensus layer (L1 layer) is too onerous
 - Both engineering and operational viewpoints
- Rollups (Layer 2) is the preferred approach for scaling
 - Arbitrum, Optimism

Proof of Stake

Proof of Work

Find **nonce** such that

$H(\text{hash}(\text{parent.header}), \text{Merkle root of tx}, \text{nonce}) < \text{threshold}$

Finding entails **work**

Nonce is the **proof**

Doing this work allows miners to **participate meaningfully** in the protocol

PoW is a Sybil and spam resistant leader election mechanism

Proof of Stake

Allow meaningful participation based on stake
block proposers own coins

Level of participation proportional to stake
higher probability of being a proposer

Doing work is replaced by owning coins
energy efficient
capital efficient – no need for mining hardware

Idea 1

PoS attempt 1

$H(\text{hash}(\text{parent.header}), \text{Merkle root of tx, public key}) < \text{threshold} \times \text{stake}$

Idea 1

PoS attempt 1

$H(\text{hash}(\text{parent.header}), \text{Merkle root of tx}, \text{public key}) < \text{threshold} \times \text{stake}$

Problem: Grinding

can try different set of tx such that Merkle root of tx works out “correctly”

so probability is not purely proportional to stake

Idea 2

PoS attempt 2

$H(\text{hash}(\text{parent.header}), \text{public key}) < \text{threshold} \times \text{stake}$

Got rid of transaction hash

Problem: Liveness

just one trial to form a block

unlike PoW where nonce can be tried at will

Idea 3

PoS attempt 3

$H(\text{hash}(\text{parent.header}), \text{timestamp}, \text{public key}) < \text{threshold} \times \text{stake}$

Idea 3

PoS attempt 3

$H(\text{hash}(\text{parent.header}), \text{timestamp}, \text{public key}) < \text{threshold} \times \text{stake}$

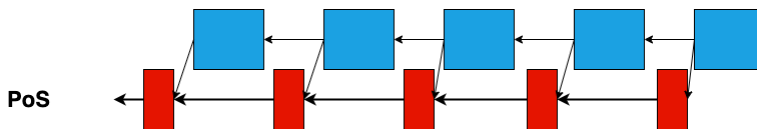
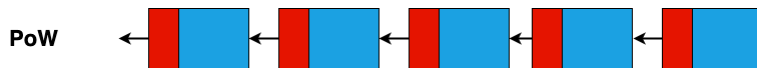
Problems:

1. Block content is not tamper-resistant against an adaptive adversary
2. Public election: vulnerable to bribery/corruption, not resistant to adaptive adversary

Idea 3

PoS attempt 3

$H(\text{hash}(\text{parent.header}), \text{timestamp}, \text{public key}) < \text{threshold} \times \text{stake}$



Block header



Block content

Resources

ECE/COS 470, Pramod Viswanath, Princeton 2024